HIGANNA

PRODUCT & TECHNICAL PLAN

Future Development Architecture

A phased roadmap for secure accounts, coins, global matchmaking, server-side bots, rewarded ads and subscriptions.

Next.js 16 | Node.js | Socket.IO | MySQL | Redis | PM2

Architecture proposal • Version 1.0 • 02 September 2026

Executive summary

Higanna should remain one full-stack application while the product grows. The code will be separated into modules, but it does not need the cost and complexity of microservices yet.

Agreed authentication: players provide only an email address and password. A safe automatic display name such as **Player4821** can be generated internally. Google and Facebook can be linked later to the same user account.

Application

Continue using the current Next.js frontend, Node.js custom server and Socket.IO realtime gateway behind Nginx and PM2.

Permanent data

Use MySQL for users, sessions, wallets, transactions, subscriptions, match records and final results.

Realtime data

Use Redis for matchmaking queues, online presence and temporary active-game state. MySQL remains the source of truth.

Security boundary

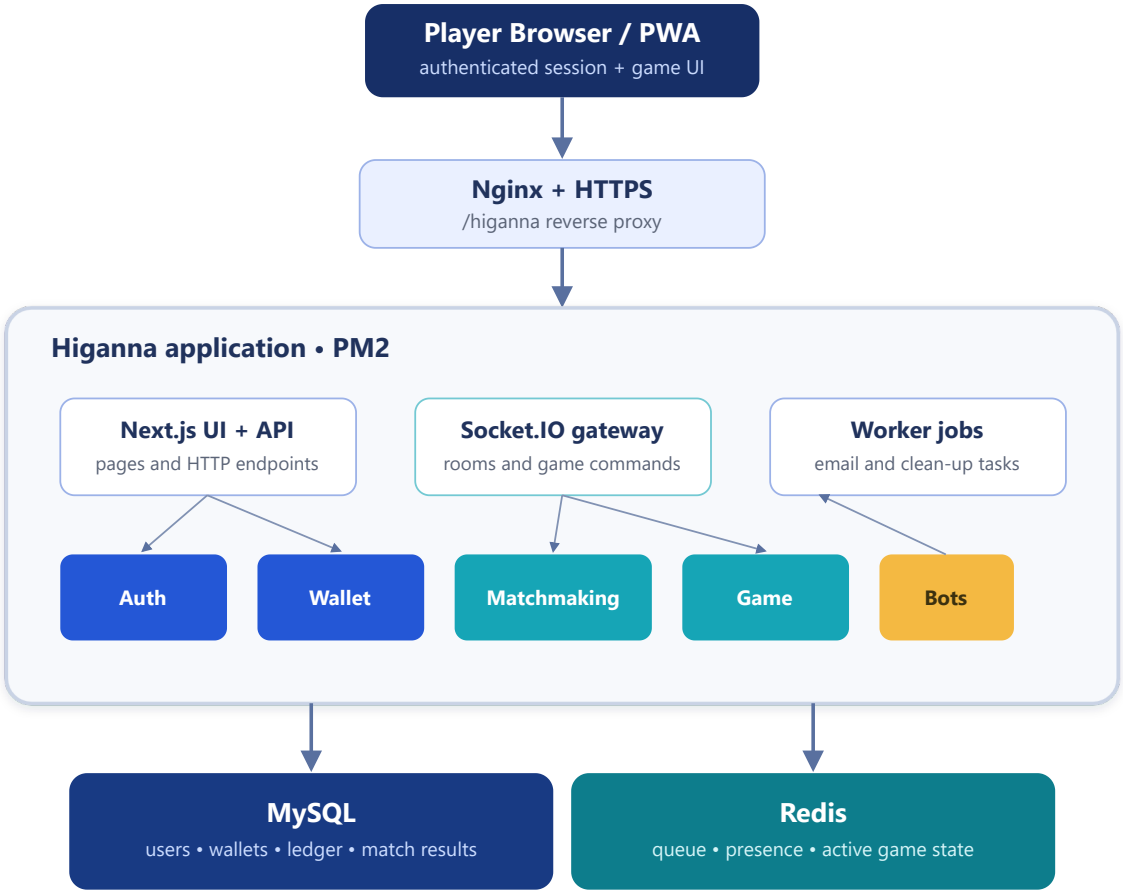
The server owns identity, coin balances, legal moves, results and rewards. The browser only sends player intentions.

Technology decisions

Area	Recommended choice	Reason
Architecture	Modular monolith	Simple deployment now, with clear module boundaries for future scaling.
Authentication	Better Auth	Email/password now; social identities, sessions, reset and verification later.
Database access	Prisma ORM	Typed MySQL queries, repeatable migrations and transaction support.
Realtime	Socket.IO	Already integrated and suitable for rooms, matchmaking and game commands.
Temporary state	Redis	Fast queue, presence, locks and multi-process coordination.
Hosting	Nginx + PM2	Preserves the current deployment model at **/higanna**.

Production prerequisite: add a real domain and HTTPS before Google/Facebook login, payments or public account launch. The current raw HTTP IP is suitable only as a temporary deployment endpoint.

Target architecture



Authentication boundary

Signup and sign-in use email/password. Secure session cookies identify HTTP and Socket.IO requests. Client-provided player IDs are not trusted.

Game boundary

The server validates turns, legal actions and final results. Bots call the same legal-action interface and receive only permitted game information.

Quick Play interpretation: a player joins a global queue; compatible human players are selected first, then server-side bots fill empty seats after a configurable timeout. Existing private-room play remains available.

Database model

Module	Tables	Responsibility
Identity	**users, accounts, sessions, verification**	Email/password accounts, future Google/Facebook identities, sessions and verification records.
Economy	**wallets, coin_ledger, coin_holds**	Available balance, immutable history and temporary match-entry reservations.
Games	**game_sessions, game_participants, game_results, game_snapshots**	Human/bot participation, completed results, statistics and crash recovery.
Matchmaking	**matchmaking_tickets**	Durable ticket status; active ordering and presence stay in Redis.
Revenue	**ad_reward_claims, subscriptions, payment_events**	Verified rewards, provider status and duplicate-event protection.

Authentication flow

Email + password › **Validate + create user** › **Secure session cookie** › **HTTP and Socket identity**

- ✓ Passwords are handled by the authentication library and never stored as plain text.
- ✓ Duplicate normalized email addresses are rejected.
- ✓ Signup/login endpoints receive validation and rate limiting.
- ✓ Password reset and email verification are added before public production launch.

Coin and matchmaking flow

Join Quick Play › **Reserve entry coins** › **Find humans** › **Fill with bots** › **Start match**

If the match starts

Capture the reserved entry coins. At completion, the server stores the result and adds rewards through the ledger.

If the match fails

Release the reservation. A retry-safe idempotency key prevents duplicate charges, refunds or rewards.

Economy rule: the frontend never sends a balance or reward amount to be trusted. All wallet changes use a MySQL transaction and append a permanent ledger entry. Initially, coins should be non-transferable and non-withdrawable.

Phased implementation

Phase 1

Database foundation

Configure MySQL, Prisma, environment variables, migrations and a reusable database connection. Confirm the existing game still builds and runs.

Phase 2

Email/password authentication

Add signup, sign-in, sign-out, sessions, validation, generated display names and protected routes. Then add verification and password reset.

Phase 3

Socket.IO authentication

Authenticate the socket handshake from the server session, remove client-controlled identity and support safe reconnection.

Phase 4

Wallet and coin ledger

Add starting balance, immutable transactions, coin reservations, entry deductions, refunds and game rewards.

Phase 5

Global Quick Play

Add Redis-backed matchmaking while preserving existing private rooms. Match humans first and fill missing seats after a timeout.

Dependency rule: each phase begins only after the previous phase meets its acceptance checks. This prevents coins and matchmaking from being built on an unverified identity layer.

Phased implementation

Phase 6

Server-side bots

Implement random legal-action and basic strategy bots through the existing engine, with difficulty and disconnect replacement.

Phase 7

Ads and subscriptions

Add provider-independent monetization, verified callbacks/webhooks, limits and duplicate-claim protection only after the wallet is stable.

Phase 8

Google and Facebook login

Link social identities to existing user accounts after a domain and HTTPS are active.

Phase 9

Production hardening

Add backups, monitoring, rate limits, security headers, staging deployment, recovery testing and documented privacy/refund rules.

First milestone: complete only MySQL + Prisma + Better Auth + signup/sign-in. Coins and matchmaking begin after authenticated HTTP and Socket.IO identity passes testing.

Milestone acceptance criteria

Account milestone

- ✓ A player registers with email/password only.
- ✓ Duplicate emails are rejected.
- ✓ The session survives a browser refresh.
- ✓ Sign-out terminates the session.
- ✓ Unauthenticated users cannot start games.

Realtime milestone

- ✓ Every socket belongs to an authenticated user.
- ✓ Client-provided user IDs are ignored.
- ✓ Reconnect returns the correct player to the game.
- ✓ Private-room behaviour remains compatible.

Wallet milestone

- ✓ Every balance change has a ledger record.
- ✓ Concurrent requests cannot overspend.
- ✓ Retries cannot duplicate a charge or reward.
- ✓ Failed matchmaking releases held coins.

Matchmaking milestone

- ✓ Real players are selected before bots.
- ✓ Bot fill begins only after the timeout.
- ✓ All actions are validated server-side.
- ✓ Results and participants persist in MySQL.

Business decisions still required

Decision	Recommended starting value
Quick Play table size	Four total seats; humans first, then bots.
Bot-fill timeout	15 seconds for the first release.
Match entry and prizes	Configure after testing game duration and retention; keep values in database settings.
Subscription benefit	Monthly coin grant plus no advertisements; avoid unlimited play until economy data is available.
Development database	Use local MySQL or a separate development database—never the live production database.

Safe release path

```
Local development → automated tests → production build
Database backup → migration plan/review → migration deploy
Upload release → build with /higanna base path → PM2 restart
Health check → authentication check → Socket.IO check → rollback if required
```

This document defines the target technical direction. Exact coin values, prize distribution, table size, bot timeout and subscription benefits remain product decisions and should be confirmed before their corresponding phase begins.